

3.12. Para cada una de las historias siguientes:

$H_1 = r_1(A); r_2(B); w_1(C); r_3(D); r_4(E); w_3(B); w_2(C); w_4(A); w_1(D);$

$H_2 = r_1(A); r_2(B); r_3(C); w_1(B); w_2(C); w_3(D);$

$H_3 = r_1(A); r_2(B); r_3(C); w_1(B); w_2(C); w_3(A);$

Se pide:

- Ubicar un *read lock* inmediatamente antes de cada acción de lectura, un *write lock* inmediatamente antes de cada acción de escritura y *unlocks* de modo que las transacciones cumplan con 2PL
- En base a lo anterior decir que acciones son denegadas y cuando ocurre un *deadlock*
- Dibujar la evolución del grafo *wait-for*
- Asumiendo un orden T_1, T_2, T_3, T_4 y un mecanismo de detección de deadlock explicar que ocurre si este mecanismo usa:
 - Un esquema Wait-Die
 - Un esquema Wound-Wait

(H1)

P) $T_1 = rl_1(A); r_1(A); wl_1(C); w_1(C);$
 $wl_1(D); w_1(D); u_1(A); u_1(C);$
 $u_1(D)$

$T_2 = rl_2(B); r_2(B); wl_2(C); w_2(C);$
 $u_2(B); u_2(C)$

$T_3 = rl_3(D); r_3(D); wl_3(B); w_3(B);$
 $u_3(D); u_3(B)$

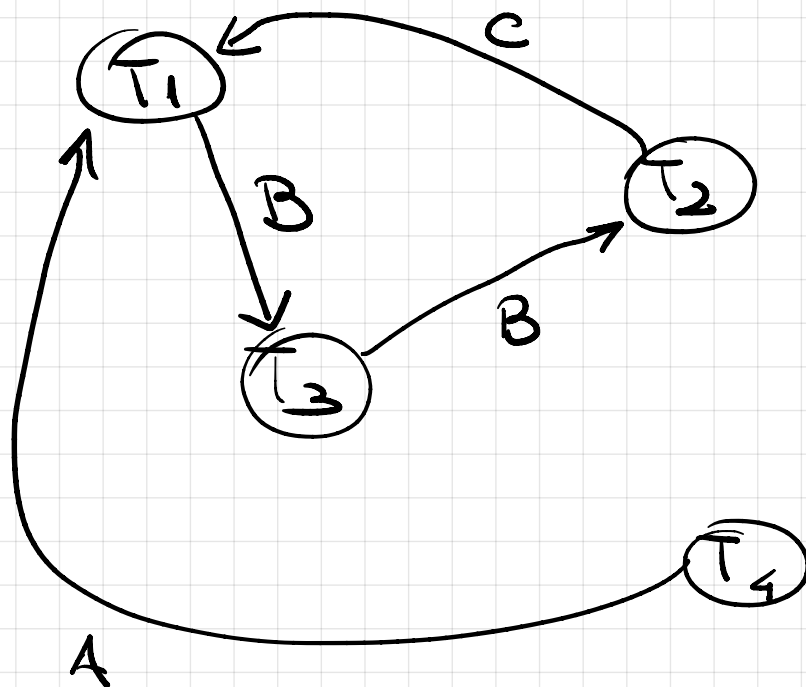
$T_4 = rl_4(E); r_4(E); wl_4(A); w_4(A);$
 $u_4(E); u_4(A)$

Co: Otro modo Tx es 2PL, por
no de serializabilidad Toda H
legal es SR (Basado en $T_1 - T_4$)

T_1	T_2	T_3	T_4
$rl_1(A)$ $r_1(A)$ ✓			
	$rl_2(B)$ $r_2(B)$ ✓		
$wl_1(c)$ $w_1(c)$ ✓			
		$rl_3(D)$ $r_3(D)$ ✓	
			$rl_4(E)$ $r_4(E)$ ✓
		$wl_3(B)$ $w_3(B)$ $\overset{T_2}{\times}$	
	$wl_2(c)$ $w_2(c)$ $\overset{T_1}{\times}$		
			$wl_4(A)$ $w_4(A)$ $\overset{T_1}{\times}$
$wl_1(D)$ $w_1(D)$ $\overset{T_3}{\times}$			

Deadlock: $T_1 \leftarrow T_2 \leftarrow T_3 \rightarrow T_1$

c)



ciclo:

$$T_1 - T_3 - T_2 - T_1$$

d)

T_1^{00}	T_2^{01}	T_3^{02}	T_4^{03}
$rl_1(A)$ $r_1(A)✓$			
	$rl_2(B)$ $r_2(B)✓$		
$wl_1(c)$ $w_1(c)✓$			
		$rl_3(d)$ $r_3(d)✓$	
			$rl_4(E)$ $r_4(E)✓$
		$wl_3(B)$ $w_3(B)$ Abort! Por T2	
	$wl_2(c)$ $w_2(c)$ Abort! Por T1		
			$wl_4(A)$ $w_4(A)$ Abort $\otimes T_1$ Por T1
$wl_1(d)$ $w_1(d)$			

El Deadlock no este más.

Si T_i intenta realizar un lock sobre un ítem y no puede porque T_j ya tiene un lock previo entonces hay dos estrategias:

Wait-Die

- Si $TS(T_i) < TS(T_j)$ (T_i más viejo que T_j), entonces T_i se lo pone en espera,
- Si $TS(T_i) > TS(T_j)$ (T_i más joven que T_j), entonces se aborta T_j (T_i dies) y se recomienza mas tarde con el mismo timestamp.

Wound-Wait

- Si $TS(T_i) < TS(T_j)$ (T_i más viejo que T_j), entonces, abortar T_j (T_i wounds T_j) y recomienza más tarde con el mismo timestamp.
- Si $TS(T_i) > TS(T_j)$ (T_i más joven que T_j), entonces, T_i se pone en espera.

Wait-Die

Wound - wait

T_1	T_2	T_3	T_4
$rl_1(A)$ $r_1(A)✓$			
	$rl_2(B)$ $r_2(B)✓$		
$wl_1(c)$ $w_1(c)✓$			
		$rl_3(D)$ $r_3(D)✓$	
			$rl_4(E)$ $r_4(E)✓$
		$wl_3(B)$ Espera a T_2	
	$wl_2(c)$ Espera a T_1		
			$wl_4(A)$ Espera a T_1
$wl_1(D)$ T_3 tiene $rl_3(D)$ Abort T_3		⊗	

$W_3(D)$

Se ve el deadlock
 T_1 , hace los
unlock siguientes.

Cómo siguen los T_1 luego del
abort, que sigue en la historia
después, aparece T_3 al final
de manera serial?